

Ingénierie des Logiciels

Dr. NGUENANG Louis Bernard

PhD en Informatique

Enseignant - Chercheur



Plan

1. **Chapitre 1 : Introduction à l'Ingénierie des Logiciels**
2. **Chapitre 2 : Les activités du Développement Logiciel**
3. **Chapitre 3 : Les Modèles de Développement Logiciel**
4. **Chapitre 4 : Les Méthodes de Développement émergentes**
5. **Chapitre 5 : Assurance qualité logicielle et les Techniques de vérification**

Chapitre 0: Introduction à l'Ingénierie des Logiciels

I- LE LOGICIEL

I.1- L'importance du logiciel

- Beaucoup d'aspects de notre quotidien ne peuvent être imaginés aujourd'hui sans logiciel.
- C'est le cas entre autres des domaines du transport, du commerce, des communications, de la médecine, des finances ou des loisirs.
- Cette omniprésence du logiciel fait que nos vies et le fonctionnement de nos sociétés dépendent fortement de sa *qualité*.
- Les erreurs logicielles peuvent causer de vrais désastres, économiques ou sanitaires.
- Un exemple emblématique est l'explosion de la première fusée Ariane 5 en 1996, qui a coûté plus d'un demi-milliard de dollars, à cause du dépassement de capacité d'une variable lors d'un calcul.

I- LE LOGICIEL

I.2 - La diversité du logiciel

- Il existe une grande variété de logiciels et de nombreuses manières de les classer.
- Une première différenciation oppose les logiciels « **génériques** », ou progiciels, qui sont vendus en grand nombre comme des produits de consommation, aux logiciels « **spécifiques** » qui sont développés pour un contexte et un client particulier.
- Une seconde différenciation repose sur la nature de la dépendance entre les logiciels et leurs environnements. Elle distingue trois classes :
 - les logiciels « autonomes » (*standalone*)
 - les logiciels qui gèrent des processus (*process support*)
 - les logiciels « embarqués » dans des systèmes (*embedded*)

I- LE LOGICIEL

I.2 - La diversité du logiciel

- Une troisième différenciation sépare les **logiciels isolés** des « **lignes de produits logiciels** ». Celles-ci regroupent une famille de logiciels présentant de fortes similarités et un même domaine d'application, comme des applications pour téléphones mobiles. Elles sont développées à partir d'éléments réutilisables (*assets*) qui peuvent être des exigences, des modèles, des codes (composants), des plans de tests, etc.
- Dans les grandes organisations, on particularise souvent ce qu'on appelle les « **logiciels du patrimoine** » (*legacy software*). Ce sont les applications les plus anciennes, complexes et peu documentées, auxquelles on touche le moins possible tant qu'elles remplissent correctement leurs fonctions en général vitales pour l'organisation.

I- LE LOGICIEL

I.2 - La diversité du logiciel

- Enfin les « logiciels web », ou « applications web », sont souvent considérés comme une catégorie à part. Ce sont des logiciels hébergés sur un serveur web et manipulables via les réseaux, Internet ou réseaux locaux, grâce à un navigateur web. On peut citer comme particularités notables :
 - les accès concurrents qu'ils subissent, avec une charge difficilement prédictible,
 - leurs exigences élevées de performance, de disponibilité et de sécurité,
 - la prédominance des données par rapport aux traitements,
 - leurs besoins fréquents d'évolution,
 - l'importance de l'ergonomie et de l'esthétique dans leur conception.
- On comprend que la notion de qualité et les exigences qui s'y attachent peuvent varier considérablement pour toute cette diversité de logiciels.

I- LE LOGICIEL

I.3 - La complexité du logiciel

- Techniquement, le terme « logiciel » désigne un riche ensemble d'artefacts incluant :
 - des codes sources et des exécutables,
 - des programmes et des données de test,
 - des fichiers de configuration,
 - des documentations « externes » à destination des utilisateurs,
 - des documentations « internes » à destination de l'équipe de développement, etc .
- **Remarques:**
 - Le développement logiciel est une activité intellectuelle à forte composante humaine (*human-intensive*),
 - Pendant le développement d'un logiciel, outre les membres de l'équipe de développement proprement dite, les autres acteurs à considérer sont le client (l'investisseur), les utilisateurs finaux et d'éventuels sous-traitants

II- LE DÉVELOPPEMENT LOGICIEL

II.1 - Les contextes du développement

- Le logiciel peut être développé dans plusieurs contextes assez différents.
 - Chez des éditeurs de logiciels qui commercialisent les logiciels qu'ils développent,
 - Dans des Sociétés de Service en Ingénierie Informatique (SSII), également nommées Entreprises de Services du Numérique (ESN), qui répondent aux besoins d'externalisation des projets informatiques, en développant des logiciels pour le compte de leurs clients,
 - En interne, dans des organisations de toutes natures, entreprises, administrations et associations, universités, etc.
 - Sur Internet, au moins en partie par des développeurs bénévoles, pour les logiciels libres.

II- LE DÉVELOPPEMENT LOGICIEL

II.2 - Les activités du développement

- Schématiquement, le développement logiciel consiste à transformer une idée ou un besoin en un logiciel fonctionnel.
- L'idée est produite par un client ou *maître d'ouvrage* (MOA).
- Le logiciel correspondant est développé par un fournisseur ou *maître d'œuvre* (MOE), puis exploité par des utilisateurs finaux.
- Lorsqu'il s'agit d'entités on parle de *Maîtrise d'ouvrage* (MOA) et de *Maîtrise d'œuvre* (MOE).
- La MOA peut être assistée en externe et en interne, souvent par d'anciens informaticiens ayant une bonne pratique de la conduite des projets : on parle de MOAD, pour *maîtrise d'ouvrage déléguée*.

II- LE DÉVELOPPEMENT LOGICIEL

II.2 - Les activités du développement

- le développement logiciel comprend en amont une phase centrée sur les *besoins* (ou exigences).
- Il s'agit pour toutes les parties prenantes d'un projet informatique de s'accorder sur l'ensemble des besoins auxquels devra répondre le système ou logiciel à construire.
- Le terme d'« ingénierie des besoins » ou « ingénierie des exigences » (*requirements engineering*) caractérise toutes les activités liées à la gestion des besoins, y compris celles qui se poursuivent tout au long du développement, comme la gestion des évolutions des besoins et de leur traçabilité.

III- LA QUALITÉ DU LOGICIEL

III.1 – Définition de la qualité logiciel

- La notion de « qualité du logiciel » n'est pas simple à définir car elle est multiforme et dépend du point de vue adopté, client ou fournisseur.
- Pour le client, le logiciel doit répondre à ses besoins (adéquation), être efficace, facile d'apprentissage et d'utilisation (ergonomie), être fiable, robuste et sûr, être peu coûteux et livré dans les délais.
- Pour le fournisseur, le coût et la durée du développement sont généralement primordiaux, de même que la facilité d'extension, de maintenance, d'adaptation et d'évolution, ainsi que la portabilité et l'interopérabilité.

III- LA QUALITÉ DU LOGICIEL

III.2 – Les critères de qualité logiciel

- La norme **ISO/CEI 9126** définit six groupes d'indicateurs de la qualité

Terme français	Terme Anglais	Signification
Capacité fonctionnelle	Functionality	La capacité d'un logiciel à répondre aux exigences fonctionnelles exprimées.
Fiabilité	Reliability	La capacité d'un logiciel à produire des résultats corrects dans des conditions précises.
Facilité d'utilisation	Usability	L'effort requis pour l'utilisation / manipulation d'un logiciel.
Efficience	Efficiency	le rapport existant entre le niveau de service d'un logiciel et la quantité de ressources utilisées, dans des conditions déterminées.
Maintenabilité	Maintainability	L'effort requis pour corriger ou ajouter des nouvelles fonctionnalités.
Portabilité	Portability	Aptitude du logiciel à être transféré d'un environnement à l'autre.

III- LA QUALITÉ DU LOGICIEL

III.2-1 – Sous-critères de qualité de Capacité Fonctionnelle

- Le critère de qualité **Capacité Fonctionnelle** regroupe les sous-critères suivant: **Aptitude, Exactitude, Interopérabilité et Sécurité.**

Caractéristiques	Sous- Caractéristiques	Signification
Capacité fonctionnelle	Aptitude	présence et adéquation d'une série de fonctions pour des tâches données
	Exactitude	fourniture de résultats ou d'effets justes ou convenus
	Interopérabilité	capacité à interagir avec des systèmes donnés
	Sécurité	aptitude à empêcher tout accès non autorisé (accidentel ou délibéré) aux programmes et données

III- LA QUALITÉ DU LOGICIEL

III.2-2 – Les Sous-critères de qualité de la Fiabilité

- Le critère de qualité **Fiabilité** regroupe les sous critères, **Maturité**, **Tolérance aux fautes**, **Possibilité de récupération**.

Caractéristiques	Sous-Caractéristiques	Signification
Fiabilité	Maturité	Fréquence des défaillances dues à des défauts du logiciel
	Tolérance aux fautes	Aptitude à maintenir un niveau de service donné en cas de défaut du logiciel ou de violation de son interface
	Possibilité de récupération	capacité à rétablir son niveau de service et de restaurer les informations directement affectées en cas de défaillance ; temps et effort nécessaire pour le faire

III- LA QUALITÉ DU LOGICIEL

III.2-3 – Les Sous-critères de qualité Facilité d'utilisation

- Le critère de qualité **Facilité d'utilisation** regroupe les sous critères, **Facilité de compréhension**, **Facilité d'apprentissage**, **Facilité d'exploitation**

Caractéristiques	Sous- Caractéristiques	Signification
Facilité d'utilisation	Facilité de compréhension	effort que doit faire l'utilisateur pour reconnaître la logique et sa mise en œuvre
	Facilité d'apprentissage	effort que doit faire l'utilisateur pour apprendre son application, il doit comprendre ce que l'on peut faire avec le logiciel, et savoir comment le faire
	Facilité d'exploitation	effort que doit faire l'utilisateur pour exploiter et contrôler l'exploitation de son application

III- LA QUALITÉ DU LOGICIEL

III.2-4 – Les Sous-critères de qualité Efficience

- Le critère de qualité **Efficience** regroupe les sous critères, **Comportement vis-à-vis du temps**, **Comportement vis-à-vis des ressources**

Caractéristiques	Sous- Caractéristiques	Signification
Efficience	Comportement vis-à-vis du temps	temps de réponses et de traitement ; débits lors de l'exécution de sa fonction
	Comportement vis-à-vis des ressources	quantité de ressources utilisées ; durée de leur utilisation lorsqu'il exécute sa fonction

III- LA QUALITÉ DU LOGICIEL

III.2-5 – Sous-critères de qualité de Maintenabilité

- Le critère de qualité **Maintenabilité** regroupe les sous-critères suivant: **Facilité d'analyse, Facilité de modification, Stabilité, Facilité de test**

Caractéristiques	Sous- Caractéristiques	Signification
Maintenabilité	Facilité d'analyse	effort nécessaire pour diagnostiquer les déficiences et causes de défaillance ou pour identifier les parties à modifier
	Facilité de modification	effort nécessaire pour modifier, remédier aux défauts ou changer d'environnement
	Stabilité	risque des effets inattendus des modifications
	Facilité de test	effort nécessaire pour valider le logiciel modifié

III- LA QUALITÉ DU LOGICIEL

III.2-6 – Sous-critères de qualité de Portabilité

- Le critère de qualité **Portabilité** regroupe les sous-critères suivant: **Facilité d'adaptation**, **Facilité d'installation**, **Conformité aux règles de portabilité**, **Interchangeabilité**

Caractéristiques	Sous-Caractéristiques	Signification
Portabilité	Facilité d'adaptation	possibilité d'adaptation à différents environnements donnés sans que l'on ait recours à d'autres actions ou moyens que ceux prévus à cet effet pour le logiciel considéré
	Facilité d'installation	effort nécessaire pour installer le logiciel dans un environnement donné
	Conformité aux règles de portabilité	conformité aux normes et aux conventions ayant trait à la portabilité
	Interchangeabilité	possibilité et effort d'utilisation du logiciel à la place d'un autre logiciel donné dans le même environnement

III- LA QUALITÉ DU LOGICIEL

III.3 – Les exigences contradictoires de qualité logiciel

- On résume parfois la variété de ce qui est attendu d'un logiciel par le sigle CQFD, pour Coûts, Qualité, Fonctionnalités, Délais.
- Cette vision simplifiée est suffisante pour montrer les tensions qui peuvent exister entre les principaux critères.
- Idéalement, il faudrait pouvoir faire du logiciel « chic et pas cher » (F élevé et C faible), ce qui n'est pas simple, et de le faire « vite et bien » (D faible et Q élevé), ce qui n'est également pas évident !
- tous ces critères sont bien entendu à pondérer en fonction du type de logiciel à développer : du logiciel critique pour la sécurité (*safety critical*), dont les défaillances peuvent avoir des conséquences désastreuses en termes humains, économiques ou environnementaux, jusqu'au logiciel « jetable », beaucoup moins exigeant.

IV- LA « CRISE DU LOGICIEL »

IV.1 – Le constat

- L'expression « **crise du logiciel** » est employée depuis la fin des années 1960 pour rendre compte de toutes les difficultés associées à sa production : délais de livraison et budgets non respectés, non-satisfaction de certains besoins des clients ou utilisateurs, difficultés d'utilisation et de maintenance, etc.
- De nombreuses études rendent compte de ce phénomène en classant les développements en trois grandes catégories :
 - **Succès** : quand le logiciel est livré à temps, sans dépassement de budget et avec toutes les fonctionnalités initialement prévues.
 - **Mitigé** : quand le logiciel est livré et opérationnel, mais avec moins de fonctionnalités qu'attendu ou un dépassement notable de budget ou des délais.
 - **Échec** : quand le logiciel est abandonné en cours de développement ou livré et non utilisé.

IV- LA « CRISE DU LOGICIEL »

IV.1 – Le constat

- Une des toutes premières études, réalisée par les militaires du Department of Defense (DoD) américain dans les années 1980, donnait des résultats absolument catastrophiques : **succès 5 %** (dont utilisation « tel que » 2 % et avec de faibles modifications 3 %), **mitigé 19 %**, **échec 76 %** (dont jamais utilisé 47 % et non livré 29 %).
- Depuis, si on se base sur les études annuelles du Standish Group, connues sous le nom de *Chaos Report* et résumées dans le tableau suivant, les progrès semblent à la fois réels et assez lents.

Année	Succès	Mitigé	Échec
1995	16 %	53 %	31 %
2000	28 %	49 %	23 %
2006	35 %	46 %	19 %
2010	37 %	42 %	21 %
2013	31 %	50 %	19 %
2014	28 %	56 %	17 %
2015	29 %	52 %	19 %

IV- LA « CRISE DU LOGICIEL »

IV.2 – L'influence de la complexité

- Les progrès, bien qu'indéniables, sont atténués car la *complexité* des applications ne cesse de s'accroître au fil des années ainsi que les exigences des utilisateurs.
- Or plus la taille s'accroît plus les échecs sont fréquents.
- Pour donner quelques ordres de grandeur à propos de la complexité des logiciels, on peut indiquer qu'un compilateur simple correspond à 20 ou 30 KLS (milliers de lignes source), un logiciel de modélisation 3D correspond à 200 KLS, le logiciel de contrôle d'un sous-marin nucléaire exige 1 000 KLS et celui de la navette spatiale plus de 2 200 KLS. **Soit un facteur 100 entre ces extrêmes**

IV- LA « CRISE DU LOGICIEL »

IV.3 – Les causes

Parmi les multiples causes avancées, beaucoup sont liées à la nature même du logiciel et donc très difficiles à combattre :

- Le logiciel est un objet *immatériel*, une structure d'information.
- Le logiciel est très *malléable*, au sens de facile à modifier, avec parfois des conséquences majeures pour des modifications infimes.
- Les défaillances et erreurs ne proviennent ni de défauts dans les matériaux ni de phénomènes d'usure dont on peut connaître les lois d'occurrence, mais d'*erreurs humaines*, peu prévisibles. **Beaucoup de ces erreurs sont difficiles à découvrir avant la livraison du produit chez le client.**

IV- LA « CRISE DU LOGICIEL »

IV.3 – Les causes

- Le logiciel ne s'use pas et ne se répare pas. Il n'y a pas de pièces de rechange ! Par contre, le logiciel peut devenir assez rapidement obsolète par rapport aux concurrents, par rapport au contexte technique, par rapport aux logiciels qui l'entourent. Il s'agit d'un domaine en *évolution* extrêmement rapide.
D'autres causes sont plus spécifiquement liées à la difficulté de comprendre les besoins des clients et des utilisateurs.
- La trop faible *implication* des clients et des utilisateurs dans les projets de développement logiciel.
- La difficulté pour les clients à *décrire leurs besoins* de façon claire et exhaustive pour ceux qui développent les applications.
- L'inévitable *évolutivité* des besoins, résultant par exemple des évolutions fréquentes dans l'environnement des applications.

IV- LA « CRISE DU LOGICIEL »

IV.3 – Les causes

Enfin, d'autres causes plus générales résultent de la difficulté à gérer des projets et des équipes, en particulier quand ils sont de grande taille, comme par exemple :

- la différence de langage et de culture entre les personnels techniques et non techniques,
- le renouvellement rapide du personnel (*turnover*)
- la difficulté à motiver les personnes sur le long terme, etc

IV- LA « CRISE DU LOGICIEL »

IV.4 – Les « mythes » du développement logiciel

Beaucoup de croyances erronées, souvent qualifiées de « **mythes** », gênent la bonne compréhension du développement logiciel et peuvent contribuer à aggraver les difficultés. Les cinq mythes suivants sont parmi les plus connus

- *Un vague énoncé est suffisant pour commencer à coder. Il sera toujours temps de voir les détails plus tard.* (En réalité, la connaissance insuffisante des besoins est une source reconnue de retards et de coûts élevés).
- *Les besoins changent sans cesse, mais ce n'est pas grave car le logiciel est flexible.* (En réalité, le coût d'une modification augmente considérablement avec l'avancement du projet).
- *Quand un programme est écrit et tourne, le travail du développeur est terminé.* (En réalité, la maintenance des applications requiert de 50 à 70 % de l'effort total).

IV- LA « CRISE DU LOGICIEL »

IV.4 – Les « mythes » du développement logiciel

- *Tant qu'un programme ne tourne pas, il n'y a aucun moyen d'en vérifier la qualité.*
(En réalité, on peut procéder à des revues ou inspections des produits du développement qui sont très efficaces pour en améliorer la qualité).
- *Si un projet prend du retard, il suffit d'ajouter des programmeurs.* (En réalité, le développement du logiciel n'est pas une activité industrielle classique. Ajouter des programmeurs peut contribuer, au contraire, à aggraver la situation). Ce mythe est énoncé dans le célèbre ouvrage *The mythical Man-Month* de Frederick Brooks.

IV- LA « CRISE DU LOGICIEL »

IV.4 – Les Solutions

- L'étude des différents éléments de solution mis en œuvre aujourd'hui pour combattre la « crise du logiciel » est au cœur du présent cours.
- **Génie Logiciel** : c'est la science et l'art de spécification, de conception, d'implémentation et d'évolution avec économie élégance et dans les délais, des programmes, documentations et les procédures opérationnelles par lesquels les ordinateurs sont rendus utiles à l'homme

V- LA MATURITÉ DES ORGANISATIONS

V.1 – Le niveau Initial.

- L'organisation ne dispose pas de procédures standardisées ni de suivi de performance. Le succès dépend essentiellement des efforts et des compétences des individus

V.2 – Le niveau Piloté

- Il y a un consensus dans l'organisation sur la manière dont les choses doivent être gérées, mais cela n'est ni formalisé ni écrit. Le rôle des chefs de projets est essentiel pour la capitalisation de l'expérience. Les coûts et les délais sont gérés.
- *Au niveau piloté*, on trouve la planification de projet, le suivi et la supervision de projet, la gestion de la sous-traitance, l'assurance qualité, la gestion de configuration.

V- LA MATURITÉ DES ORGANISATIONS

V.2 – Le niveau Piloté

- *Au niveau piloté*, on trouve aussi la gestion des exigences qui implique:
 - la compréhension et la prise en compte des exigences par les développeurs,
 - la gestion des évolutions des exigences et leur traçabilité,
 - la conformité des produits du projet aux exigences.

V.3 – Le niveau Standardisé

- Le processus de développement est formalisé, documenté et appliqué. La capitalisation de l'expérience est centralisée. Une structure chargée de la qualité et des méthodes est en place.
- *Au niveau standardisé*, on trouve la définition du processus organisationnel, la focalisation sur le processus organisationnel, la formation à l'organisation, les revues par les pairs, la coordination entre les groupes.

V- LA MATURITÉ DES ORGANISATIONS

V.3 – Le niveau Standardisé

- *Au niveau standardisé, on trouve aussi le développement des exigences qui implique :*
 - l'élucidation des souhaits et des besoins des clients et leur expression en exigences client,
 - la dérivation des exigences techniques sur l'application, ses composants et son interface,
 - l'analyse et la validation des exigences.

V.4 – Le niveau Quantifié

- Les projets sont pilotés sur la base d'indicateurs objectifs de qualité des produits et des processus. Un processus formel de collecte des informations est en place.

V- LA MATURITÉ DES ORGANISATIONS

V.4 – Le niveau Quantifié

- *Au niveau quantifié*, on trouve la gestion quantitative des processus et de la qualité logicielle.

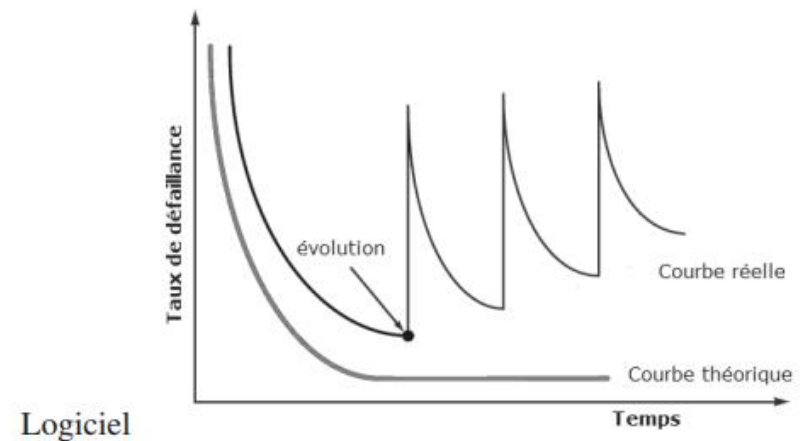
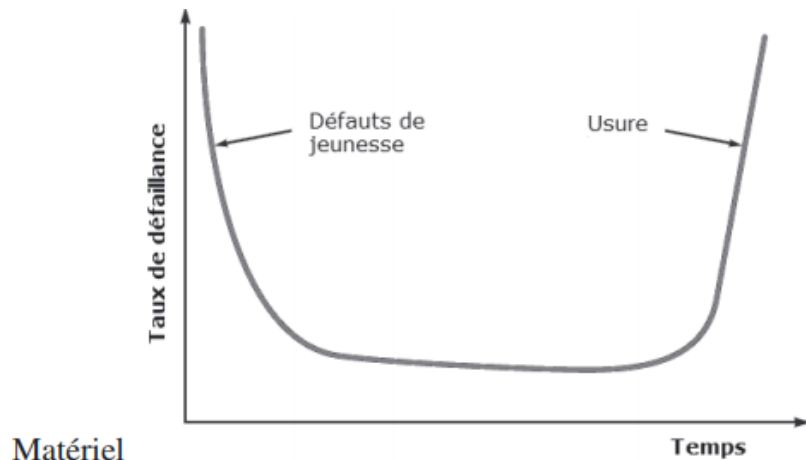
V.5 – Le niveau Optimisé

- L'amélioration continue des processus devient une préoccupation centrale. L'organisation se donne les moyens d'identifier et de mesurer les faiblesses de ses processus. Une cellule de veille identifie puis met en œuvre les technologies innovantes et les pratiques d'ingénierie logicielle les plus efficaces.
- *Au niveau optimisé*, on trouve la gestion des changements technologiques, l'innovation organisationnelle et la prévention des défauts,

VI- Exercices

Exercice 1 : Les courbes de défaillance du matériel et du logiciel

- Commenter les courbes suivantes, qui donnent l'évolution au cours du temps du nombre de défaillances respectivement du matériel et du logiciel.



VI- Exercices

Exercice 2 : La diversité des applications

Donner les principales qualités attendues :

- a) d'un site de commerce électronique,
- b) d'un système temps réel (par exemple de guidage au sol d'une fusée),
- c) d'un système embarqué (par exemple de contrôle du freinage d'une voiture).

Exercice 3 : *The mythical man-month*

- L'homme-mois est souvent utilisé comme unité de mesure de l'effort de développement. Frederick Brooks a critiqué cette unité dans son ouvrage *The mythical Man-Month*.

VI- Exercices

Exercice 3 : *The mythical man-month*

- L'hypothèse qu'un homme pendant deux mois équivaut à deux hommes pendant un mois, les deux valant deux hommes-mois, est le plus souvent fausse. Expliquer pourquoi en commentant les figures suivantes.

